Student notes — Session 13 · Multidimensional arrays

Slides: [`Class_13_Multidimensional_arrays.pdf`](../slides/Class_13_Multidimensional_arrays.pdf). These notes are the notes pane of the student copy of the deck.

Slide 1 — Quiz 3: 25 minutes

This slide was on the screen during Quiz 3. Quiz 3 covered classes 7 to 10. Your marked paper comes on Blackboard: Grades, Quiz 3, the feedback.

Quiz 4 is on **Friday 23 October** and covers classes 11 and 12: characters and strings.

Slide 2 — Two-dimensional arrays

The programs of this class are on Blackboard: Week 7 module, *Class 13 code*. It is a zip file. Unzip it and open the folder in Visual Studio Code. Every file has a comment at the top: what the program does, how to run it, and a list called **Try** with small changes to make. Make them.

How to run a program. Same as every week. In the terminal type `gcc -Wall name.c -o name` and press Enter, then `./name` (or `.\name` in Windows PowerShell) and press Enter.

Today: tables of numbers. A table has rows and columns. In C it is a **two-dimensional array**.

Slide 3 — What you know about arrays

What you need from classes 9 and 10:

- `int day[4];` makes 4 boxes: `day[0]`, `day[1]`, `day[2]`, `day[3]`.
- `int day[4] = {3, 5, 8, 4};` fills them.
- A `for` loop visits every box: `c < 4`, never `c <= 4`.
- C does not check the index. You must.

Slide 4 — Recap: what is on the screen?

Try to answer before you look at the next slide.

Slide 5 — Recap: what is on the screen?

The screen shows `3 5 8 4`. `c` goes 0, 1, 2, 3, and the loop stops when `c` is 4. The file is `01_recap_row.c`.

Slide 6 — One row is not enough

Now you have three days, and four temperatures each day: 12 numbers.

- Twelve variables are too many.
- Three arrays `day0`, `day1`, `day2` are better, but a loop cannot go through the names.
- One **table** with 3 rows and 4 columns is the answer.

Slide 7 — Today's roadmap

Five parts:

1. **A table**: `int t[3][4];`
2. **Two loops**, one inside the other.
3. **Row sums** and **column sums**.

4. **Read** a table, and two **careful** cases.
5. **Your turn**, and **Project 2**.

Slide 8 — A table

Part 1: a table. The rule of the day: **rows first, then columns**.

Slide 9 — int t[3][4];

```
int t[3][4];
```

- The first number is the number of **rows**: 3.
- The second number is the number of **columns**: 4.
- 3 × 4 = 12 boxes, all `int`.

One box is `t[row][column]`. Both numbers start at 0. So the rows are 0, 1, 2 and the columns are 0, 1, 2, 3.

Slide 10 — The table t

The table as a picture:

	column 0	column 1	column 2	column 3
row 0	3	5	8	4
row 1	2	6	9	1
row 2	-1	0	4	2

`t[1][2]` means: go to row 1, then to column 2. It is **9**.

Slide 11 — Fill it with a list

```
int t[3][4] = {
    {3, 5, 8, 4},
    {2, 6, 9, 1},
    {-1, 0, 4, 2}
};
```

- One pair of braces `{ }` for each row.
- A comma between the rows.
- A semicolon after the last brace.

Slide 12 — Predict: three boxes

Try to answer before you look at the next slide. Use the picture of slide 10.

Slide 13 — Predict: three boxes

The program prints:

```
3
9
2
```

`t[0][0]` is the first box: 3. `t[1][2]` is row 1, column 2: 9. `t[2][3]` is the last box: 2. The file is `02_table.c`.

Slide 14 — Careful: the order

Try to answer before you look at the next slide.

Slide 15 — Careful: the order

No, they are different boxes.

- `t[1][2]` is row 1, column 2: **9**.
- `t[2][1]` is row 2, column 1: **0**.

The first index is always the row.

Slide 16 — Missing values become 0

```
int t[2][3] = {
    {1},
    {4, 5}
};
```

Every box you do not give a value becomes 0. Printed row by row:

```
1 0 0
4 5 0
```

So `int t[2][3] = {0};` makes every box 0. Without any list, the boxes hold garbage. The file is `03_missing_zero.c`.

Slide 17 — Two loops

Part 2: to visit every box of a table, you need two loops, one inside the other.

Slide 18 — Visit every box

```
for (int r = 0; r < 3; r++) {
    for (int c = 0; c < 4; c++) {
        printf("(%d,%d) ", r, c);
    }
    printf("\n");
}
```

- The **outer** loop picks the row: `r` is 0, 1, 2.
- The **inner** loop walks along every column of that row: `c` is 0, 1, 2, 3.

The output shows the boxes in the order the loops visit them:

```
(0,0) (0,1) (0,2) (0,3)
(1,0) (1,1) (1,2) (1,3)
(2,0) (2,1) (2,2) (2,3)
```

The file is `04_order.c`.

Slide 19 — The inner loop runs to the end first

For every row, the inner loop runs all the way: `c` is 0, 1, 2, 3. Only then does `r` go up by one, and `c` starts again at 0. So after `(0,3)` comes `(1,0)`.

3 rows × 4 columns: the line inside both loops runs **12 times**, once for each box.

Slide 20 — Print the whole table

```
for (int r = 0; r < 3; r++) {
    for (int c = 0; c < 4; c++) {
        printf("%4d", t[r][c]);
    }
    printf("\n");
}
```

`%4d` prints every number in 4 places, so the columns line up. After each row, `printf("\n")` starts a new line.

```
   3   5   8   4
   2   6   9   1
  -1   0   4   2
```

The file is `05_print_grid.c`.

Slide 21 — Write the sizes once: #define

```
#define ROWS 3
#define COLS 4
```

`ROWS` and `COLS` are names for 3 and 4. Write them at the top, and use the names everywhere: `int t[ROWS][COLS];`, `r < ROWS`, `c < COLS`. To change the size, you change one line, and every loop follows. There is no semicolon after a `#define`.

Slide 22 — Labels on the board

First one loop prints the column numbers on top. Then every row starts with its row number and a bar:

```
        0   1   2   3
0 |     3   5   8   4
1 |     2   6   9   1
2 |    -1   0   4   2
```

With labels, the player can say "row 2, column 3" without counting. You need this in Project 2. The file is `06_define_labels.c`.

Slide 23 — Fill it with a loop

You can fill a table with loops instead of a list:

```
int m[ROWS][COLS];
for (int r = 0; r < ROWS; r++) {
    for (int c = 0; c < COLS; c++) {
        m[r][c] = (r + 1) * (c + 1);
    }
}
```

With `ROWS` 4 and `COLS` 5, this makes a small multiplication table:

```
 1   2   3   4   5
 2   4   6   8  10
 3   6   9  12  15
 4   8  12  16  20
```

The file is `07_fill.c`.

Slide 24 — Predict: one box

Try to answer before you look at the next slide.

Slide 25 — Predict: one box

`m[2][3]` is **12**: (2 + 1) × (3 + 1) = 3 × 4. You can find it in the output: row 2, column 3.

Slide 26 — Row and column sums

Part 3: sums. Where the sum starts matters.

Slide 27 — The sum of every row

```
for (int r = 0; r < ROWS; r++) {
    int sum = 0;
    for (int c = 0; c < COLS; c++) {
        sum += t[r][c];
    }
    printf("row %d: %d\n", r, sum);
}
```

`int sum = 0;` is **inside** the outer loop. So every row starts with a fresh sum. After the inner loop, the sum of that row is printed:

```
row 0: 20
row 1: 18
row 2: 5
```

The file is `08_row_sums.c`.

Slide 28 — Row 0, step by step

Row 0 is 3, 5, 8, 4. The sum starts at 0.

c	t[0][c]	sum
-	-	0
0	3	3
1	5	8

Slide 29 — Row 0, step by step

c	t[0][c]	sum
-	-	0
0	3	3

c	t[0][c]	sum
1	5	8
2	8	16
3	4	20

Then `printf` prints `row 0: 20`. For row 1, `sum` starts again at 0.

Slide 30 — Careful: where does sum start?

Try to answer before you look at the next slide.

Slide 31 — Careful: where does sum start?

If `int sum = 0;` is above the outer loop, the sum never starts again:

```
row 0: 20
row 1: 38
row 2: 43
```

Row 1 adds its 18 to the 20 of row 0. Row 2 adds 5 more. Row 0 is still right, so this bug is easy to miss. Try it: it is Try 3 in `08_row_sums.c`.

Slide 32 — The sum of every column

```
for (int c = 0; c < COLS; c++) {
    int sum = 0;
    for (int r = 0; r < ROWS; r++) {
        sum += t[r][c];
    }
    printf("column %d: %d\n", c, sum);
}
```

Now the **outer** loop is the column, and the inner loop goes down the rows. The box is still `t[r][c]`: row first.

```
column 0: 4
column 1: 11
column 2: 21
column 3: 7
```

The file is `09_col_sums.c`.

Slide 33 — Rows or columns: swap the loops

sum of every	outer loop	inner loop	the box
row	`r < ROWS`	`c < COLS`	`t[r][c]`
column	`c < COLS`	`r < ROWS`	`t[r][c]`

Only the order of the loops changes. The box is always `t[r][c]`.

Slide 34 — Read, and be careful

Part 4: reading a table from the keyboard, and two mistakes C does not catch.

Slide 35 — Read a table with scanf

```
for (int r = 0; r < ROWS; r++) {
    for (int c = 0; c < COLS; c++) {
        scanf("%d", &t[r][c]);
    }
}
```

One box is one `int`, so `scanf` needs the `&`. Row 0 is filled first, then row 1. Here `ROWS` is 2 and `COLS` is 3:

```
Type 6 numbers:
1 2 3
4 5 6
   1   2   3
   4   5   6
```

You can also type all six numbers on one line. The file is `10_read_grid.c`.

Slide 36 — Predict: column 4

Try to answer before you look at the next slide.

Slide 37 — It prints 2

The columns are 0 to 3, so `t[0][4]` does not exist. But the program compiles with **no error and no warning**, and prints `2`.

That 2 is `t[1][0]`, the first box of row 1. The file is `11_past_the_row.c`.

Slide 38 — The rows sit one after another

In memory, the table is one long line of boxes: row 0, then row 1, then row 2.

`[0][0]`	`[0][1]`	`[0][2]`	`[0][3]`	`[1][0]`	`[1][1]`	`…`
3	5	8	4	2	6	...

One box past the end of row 0 is the first box of row 1. It is still a bug: never use an index outside the table.

Slide 39 — One row too far

```
for (int r = 0; r <= ROWS; r++) {
    printf("%d ", t[r][0]);
}
```

With `<=`, `r` takes 0, 1, 2 **and 3**. There is no row 3. The first three numbers are right; the last one is garbage, a different number on every run. Two runs:

```
3 2 -1 943716464

3 2 -1 -136316352
```

gcc gives no warning. The file is `12_bug_le_rows.c`.

Slide 40 — The rule for indexes

- Rows go from `0` to `ROWS - 1`. Columns go from `0` to `COLS - 1`.
- The loops say `r < ROWS` and `c < COLS`. Never `<=`.

- C does not check. A wrong index reads some other box, or garbage.

Slide 41 — A table of char

A table can hold characters too:

```
char g[ROWS][COLS];
```

Two loops put `'.'` in every box. Then `g[1][3] = '*';` puts a star in row 1, column 3. Two more loops print it with `%c`. Here `ROWS` is 3 and `COLS` is 5:

```
. . . . .
. . . * .
. . . . .
```

The file is `13_char_grid.c`.

Slide 42 — Your turn

Part 5: two short tasks. Write them on paper or in VS Code before you look at the answer.

Slide 43 — Your turn: below zero

Use the table `t` of today. Count the boxes below 0. Print the row and column of each one.

Slide 44 — Below zero: one answer

```
int count = 0;
for (int r = 0; r < ROWS; r++) {
    for (int c = 0; c < COLS; c++) {
        if (t[r][c] < 0) {
            printf("below zero at (%d,%d)\n", r, c);
            count++;
        }
    }
}
printf("%d below zero\n", count);
```

The count starts once, before both loops, and is printed after both. Output:

```
below zero at (2,0)
1 below zero
```

The file is `14_task_below_zero.c`.

Slide 45 — Your turn: the largest of each row

Print the largest value of every row of `t`. Use the pattern of class 10: the largest so far starts at the first box.

Slide 46 — Largest of each row: one answer

```
for (int r = 0; r < ROWS; r++) {
    int max = t[r][0];
    for (int c = 1; c < COLS; c++) {
```

```
            if (t[r][c] > max) {
                max = t[r][c];
            }
        }
        printf("row %d: max %d\n", r, max);
    }
```

`max` starts at `t[r][0]`, the first box of the row, inside the outer loop: a fresh start for every row. Output:

```
row 0: max 8
row 1: max 9
row 2: max 4
```

Why not `max = 0`? A row of only negative temperatures would then print 0, a number that is not in the row. The file is `15_task_row_max.c`.

Slide 47 — Today in one slide

- `int t[3][4];` is a table: 3 rows, 4 columns, 12 boxes.
- `t[r][c]`: row first, then column. Both start at 0.
- Two loops visit every box. The outer loop picks the row.
- For row sums, `sum = 0` goes inside the outer loop.
- Use `r < ROWS` and `c < COLS`. C does not check the index.

Slide 48 — Project 2: Treasure Hunt

Project 2 is a game, **Treasure Hunt**:

- A hidden board of 8 × 8 cells. 5 treasures and 3 traps are hidden on it.
- You pick a row and a column to look at one cell. You win when you find all 5 treasures. You have 3 lives, and every trap takes one.
- Your program uses two tables: one for what is in each cell, one for what the player has already seen.

Today's loops print the board with labels. On Friday you learn how to look at the cells around a cell, which the game needs for its clues.

Slide 49 — Project 2: what and when

- The brief is on Blackboard, in the **Projects** folder. Read all of it this week.
- You hand in one file: `treasure_hunt.c`.
- It is due **Monday 9 November, 23:59**.
- 50 points for the program, 50 points for a live talk with a TA about your own code. A program you cannot explain does not get the talk's points.

Slide 50 — Before Friday

- **Knowledge check 13** is on Blackboard. It is about today's slides.
- **Quiz 4 is on Friday 23 October.** It covers classes 11 and 12: characters and strings. A TA runs Friday's class.
- **Midterm: Friday 30 October.** It covers classes 1 to 14.

- Today's code is in the Week 7 module: **Class 13 code**.

Next class: tables sent to functions, matrices, and the cells around a cell.